



VL

STUDENT READER · COURSE NOTES

MLOps & Model Deployment

Concepts, examples and insights for all ten sessions — the companion to the Lab Manual.

CONTENTS

Course Reader

01 — Introduction to MLOps	3
02 — Data & Feature Management	7
03 — Experiment Tracking with MLflow	12
04 — Model Serialization & Packaging	16
05 — Serving Models with FastAPI	20
06 — Containerization with Docker	24
07 — CI/CD for Machine Learning	28
08 — Deployment to a Free Host	32
09 — Monitoring, Drift & Governance	36
10 — Capstone & Interview Prep	40
MLOps Cheat Sheet — One-Page Quick Reference	44

SESSION 1

01 — Introduction to MLOps

Learning objectives

By the end of this session you should be able to: - Describe the end-to-end machine learning (ML) lifecycle and explain why most models fail *after* training, not during it. - Explain the idea of **technical debt** in ML systems and name at least three ML-specific sources of it. - Distinguish MLOps **maturity levels 0, 1, and 2** and place a real project at the right level. - Name the core team roles (Data Scientist, ML Engineer, Data Engineer, SRE) and describe how work flows between them.

Key terms

- **MLOps** — the set of practices for reliably building, deploying, monitoring, and maintaining ML systems in production. Think "DevOps for ML, plus data."
- **ML lifecycle** — the repeating loop from problem framing to data to training to deployment to monitoring and back.
- **Technical debt** — the future cost you take on when you choose a quick solution now instead of a clean one.
- **Training-serving skew** — when the data or code path used to train a model differs from the one used to serve it.
- **Feedback loop** — when a model's own predictions influence the future data it is trained on.
- **Maturity level** — how automated and repeatable your ML process is (0 = manual, 2 = fully automated).

Why MLOps exists

In a university ML course you spend 95% of your time on the model: pick an algorithm, tune hyperparameters, push accuracy up. In industry that model is maybe **10% of the real system**. The famous Google paper "*Hidden Technical Debt in Machine Learning Systems*" (Sculley et al., 2015) drew a diagram where the ML code is a tiny box surrounded by huge boxes: data collection, feature engineering, serving infrastructure, monitoring, configuration. MLOps is about *all the other boxes*.

A model that scores 92% accuracy in your notebook is worthless if: - nobody can reproduce it next month, - it can't answer a live HTTP request in under a second, - it silently gets worse as the world changes and no one notices.

MLOps is the discipline that turns a `model.pkl` on your laptop into a service people can depend on.

The ML lifecycle

- | | |
|-----------------------|--|
| 1. Problem framing | -> What decision does this model support? What metric matters? |
| 2. Data | -> collect, clean, version, validate |
| 3. Feature pipeline | -> transform raw data into model inputs |
| 4. Train + experiment | -> try models, track params/metrics (MLflow) |
| 5. Evaluate | -> offline metrics + business checks |
| 6. Package | -> serialize model + pin dependencies |
| 7. Deploy | -> serve behind an API (FastAPI in a Docker container) |
| 8. Monitor | -> latency, errors, data drift, accuracy |
| 9. Retrain | -> new data triggers a new loop back to step 2 |

Why this matters: Notice the arrow from step 8 back to step 2. Traditional software is "build once, patch bugs." ML is a *loop* — the world changes, so your model must too. If your process can't cheaply go around this loop, you don't have an ML system, you have a one-time science experiment.

Why models fail after training

Training is a controlled environment: fixed dataset, fixed code, offline. Production is hostile. The common failure modes:

1. **The world changed (drift).** You trained a loan-default model on 2023 data; interest rates jumped in 2024 and borrower behavior shifted. The model is still confident — and wrong. (Covered in depth in note 09.)
2. **Training-serving skew.** In your notebook you computed `age = 2024 - birth_year`. In the live API someone computes `age` in months. Same model, different inputs, garbage predictions.
3. **Data pipeline breakage.** An upstream team renames a column from `price_usd` to `price`. Your feature code silently fills nulls with 0. Predictions rot.
4. **No reproducibility.** Six months later a prediction is disputed. You can't rebuild the exact model that made it because the data, code, and library versions are all gone.

```
# Training time (notebook)
df["income_k"] = df["income"] / 1000    # income in thousands

# Serving time (API written by someone else, months later)
income_k = request.income    # oops - already in raw rupees/dollars
prediction = model.predict([[income_k, ...]]) # skew -> silently wrong
```

Why this matters: This bug throws **no error**. The API returns 200 OK, the model returns a number, dashboards look green. Skew is the most dangerous ML bug precisely because everything *appears* to work. The whole point of MLOps tooling (feature pipelines, schema validation) is to make skew impossible or loud.

Technical debt in ML systems

Regular code debt is bad. ML debt is worse because it hides in **data**, not just code. Key ML-specific sources:

- **Hidden feedback loops.** A recommender shows users item A more, so users click A more, so the next training set says "everyone loves A." The model created its own truth. Two models can even influence each other indirectly through shared users.
- **Glue code.** 90% of a real ML repo is code shuffling data into and out of a general-purpose library — reformatting, joining, patching. It is brittle and rarely tested.
- **Data dependencies cost more than code dependencies.** In code you can use a compiler/linter to find unused imports. There is no compiler that tells you a feature is no longer worth its risk. **Undeclared consumers** are a related trap: a model's output gets read by three other teams you don't know about, so you can never safely change it.
- **Pipeline jungles / dead experimental code paths.** Months of `if experiment_v3:` branches nobody dares delete.
- **Configuration debt.** Dozens of flags (learning rate, feature toggles, thresholds) with no validation. A wrong config is as damaging as a wrong line of code.

Instructor insight: When I review a struggling ML project at ConverSight, the model is almost never the problem. It's a hand-run notebook, a CSV emailed between two people, and a feature computed one way in training and another in production. Fix the *process* and the accuracy problems often disappear on their own. Debt is invisible until the day you must move fast — then it's the only thing that matters.

MLOps maturity levels

Think of these as a ladder. Most student projects and many startups live at Level 0. You climb only when the pain justifies the cost.

- **Level 0 — Manual process.** Everything by hand. Data scientist runs a notebook, exports `model.pkl`, emails it to an engineer who wraps it in an API. Few releases per year. No automated tests, no monitoring. *Fine for a proof of concept; dangerous for anything real.*
- **Level 1 — ML pipeline automation.** The *training pipeline* is automated and can be triggered on new data. You have reproducible steps, experiment tracking, and a model registry. Retraining is a button, not a person's afternoon. Continuous *training* (CT) appears here.
- **Level 2 — Full CI/CD automation.** The *pipeline itself* is built, tested, and deployed automatically. Push code -> tests run -> pipeline rebuilds -> new model trains, is validated, and is deployed with monitoring, all automatically. This is where big teams shipping models daily operate.

```
Level 0:  human ----> notebook ----> pkl file ----> "here you go, deploy this"
Level 1:  new data --> [automated training pipeline] --> registered model --> deploy
Level 2:  git push  --> [CI/CD builds & tests pipeline] --> [auto train+validate] --> auto
          deploy + monitor
```

Why this matters: In this course we deliberately build a **Level 1-ish** system with free tools: MLflow for tracking/registry, FastAPI + Docker for serving, and GitHub Actions for CI/CD (a taste of Level 2). You do not need paid cloud or a big team to practice real MLOps. You need discipline and the right free tools.

Team roles and workflow

Real ML is a relay race, not a solo sprint:

- **Data Engineer** — owns pipelines that deliver clean, reliable, versioned data. If the data is late or wrong, nothing downstream matters.
- **Data Scientist (DS)** — frames the problem, explores data, builds and evaluates models, runs experiments. Optimizes for *model quality*.
- **ML Engineer** — takes the DS's model and makes it production-grade: packaging, APIs, containers, pipelines, tests. Optimizes for *reliability and reproducibility*. (In small teams the DS and ML Engineer are the same person — often you.)
- **SRE / Platform Engineer** — keeps the deployed service alive: uptime, scaling, alerting, on-call. Optimizes for *availability*.

A healthy handoff looks like: Data Engineer guarantees a versioned dataset -> DS trains and registers a model in MLflow -> ML Engineer packages it and wires CI/CD -> SRE monitors it in production and raises an alert when drift or errors spike -> everyone loops back.

Watch out — the "toss it over the wall" antipattern. The classic failure is a DS who finishes a notebook and throws the `.pkl` at engineering with no environment info, no metrics, no data version. Engineering

can't reproduce it, so they hard-code hacks to make it run, creating glue code and skew. **Fix:** the DS's deliverable is never just a model file — it's a *registered, versioned model plus the exact data and environment that produced it*. That single habit prevents most of the debt described above.

Mini-FAQ

Q: Is MLOps just DevOps with a new name? A: No. DevOps versions *code*. MLOps must version code **plus data plus model**, and it adds two problems DevOps never had: models silently degrade as the world changes (drift), and training-serving skew. Same spirit, harder problem.

Q: We're a two-person student team. Do we really need all this? A: You need the *habits*, not the heavy tooling. Pin your dependencies, set random seeds, track experiments in MLflow, and version your data. That's maybe an hour of setup and it's the difference between "we can reproduce our best result" and "we lost it."

Q: What single practice gives the biggest payoff first? A: Reproducibility. If you can rebuild any past model exactly (same data + code + libraries), you can debug, compare, and trust everything else. Start there — it's the foundation for note 02.

SESSION 2

02 — Data & Feature Management

Learning objectives

By the end of this session you should be able to: - Explain why **data versioning** is as important as code versioning, and describe a simple way to do it for free. - Build the mental model of a **feature pipeline** and identify where **train/serve skew** creeps in. - Make an experiment **reproducible** with pinned dependencies and fixed random seeds. - Describe conceptually what a **feature store** is and the problem it solves.

Key terms

- **Data versioning** — snapshotting a dataset so you can retrieve the exact bytes used for a given model.
- **Reproducibility** — the ability to get the same result again from the same inputs.
- **Feature** — a single input signal to a model (e.g., `avg_purchase_last_30d`).
- **Feature pipeline** — the code that turns raw data into features.
- **Train/serve skew** — a mismatch between features computed at training time and at serving time.
- **Feature store** — a central system that computes, stores, and serves features consistently to both training and serving.
- **Seed** — a fixed number that makes "random" operations deterministic.

Data is code you can't see

You already version code with Git. But your model is a function of **code AND data**. Change either and you get a different model. So if you version only code, half your experiment is untracked. When someone asks "why did accuracy drop from last week's model?" and you can't say whether the code or the data changed, you're stuck.

The minimum viable habit: for every model, record **which dataset produced it**. Options, cheapest first:

1. **Content hash + a manifest**. Compute a hash of the dataset file and log it alongside the model. If the hash matches, the data is byte-identical.
2. **Immutable, dated snapshots**. Never overwrite `data.csv`. Save `data_2026-07-17.parquet` and treat it as read-only.
3. **DVC (Data Version Control)** — a free tool that works like "Git for data": it stores small pointer files in Git and the big data in normal storage. `dvc add data.csv` then `git commit` the `.dvc` pointer.

```
import hashlib, json, pathlib

def file_hash(path):
    h = hashlib.sha256()
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(8192), b''):
            h.update(chunk)
    return h.hexdigest()

manifest = {
    "dataset": "customers_2026-07-17.parquet",
    "sha256": file_hash("customers_2026-07-17.parquet"),
    "rows": 48211,
    "created": "2026-07-17",
}

pathlib.Path("data_manifest.json").write_text(json.dumps(manifest, indent=2))
```

Why this matters: This 12-line script — no paid tools — lets you *prove* which data made a model. When a prediction is disputed months later, you point to the hash. When accuracy drops, you check whether the hash changed. Data versioning is the cheapest insurance in ML.

Feature pipelines

A **feature pipeline** is the code that transforms raw records into the numeric inputs a model expects. Example: from a raw `transactions` table you might compute `num_orders_30d`, `avg_order_value`, `days_since_last_order`.

Two critical properties of a good feature pipeline: - **Deterministic** — same input always yields the same feature. - **Shared** — the *same code* runs at training time and at serving time.

That second point is the whole ballgame.

```
# features.py -- ONE function, imported by BOTH the trainer and the API
import pandas as pd

def build_features(raw: pd.DataFrame) -> pd.DataFrame:
    out = pd.DataFrame()
    out["income_k"] = raw["income"] / 1000.0
    out["age"] = 2026 - raw["birth_year"]
    out["orders_per_year"] = raw["total_orders"] / raw["tenure_years"].clip(lower=1)
    return out[["income_k", "age", "orders_per_year"]] # fixed column order!
```

```
# train.py
from features import build_features
X = build_features(raw_train) # same function

# serve.py (FastAPI)
from features import build_features
X = build_features(one_row_dataframe) # SAME function -> no skew
```

Why this matters: By putting feature logic in *one* imported module used by both training and serving, you make skew structurally impossible for those features. Copy-pasting the transformation into the API is the single most common cause of production ML bugs.

Train/serve skew — the recurring villain

Skew happens whenever the features a model sees in production differ from what it saw in training. Sources:

1. **Duplicated logic** — feature computed twice, in two places, that drift apart.
2. **Different time windows** — training used "last 30 days" of history; serving only has "last 7 days" available live.
3. **Missing-value handling** — training dropped nulls; serving fills them with 0.
4. **Column order / dtype** — model expects `[income_k, age, orders]`; API sends `[age, income_k, orders]`. Many models won't error — they'll just be wrong.

```
# Defensive check: assert the columns and order match what the model was trained on
EXPECTED = ["income_k", "age", "orders_per_year"]
```

```
def validate(features: pd.DataFrame):
    assert list(features.columns) == EXPECTED, f"Feature skew! got {list(features.columns)}"
    assert not features.isnull().any().any(), "Nulls in features at serve time"
```

Watch out: The scariest skew bugs are *silent*. A tree model happily accepts columns in the wrong order and returns a confident, wrong answer with a 200 OK. Always assert the schema (names, order, dtypes, no unexpected nulls) at the serving boundary. Loud failures beat silent wrong answers.

Reproducible environments

Reproducibility = same **data** + same **code** + same **environment** + same **seeds**. We covered data. Now the environment.

Pin your dependencies. `pip install scikit-learn` today gives a different version than six months ago, and library upgrades change results (and sometimes break loading old models). Pin exact versions:

```
# requirements.txt -- exact versions, not ranges
scikit-learn==1.4.2
pandas==2.2.1
numpy==1.26.4
mlflow==2.11.3
fastapi==0.110.0
```

```
# Freeze whatever you currently have installed
pip freeze > requirements.txt
# Reproduce later, anywhere
pip install -r requirements.txt
```

Fix your seeds. "Random" splits, shuffles, and weight initializations are pseudo-random. Fix the seed and they become deterministic.

```
import numpy as np, random
SEED = 42
random.seed(SEED)
np.random.seed(SEED)

from sklearn.model_selection import train_test_split
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=SEED)
# For deep learning also set torch.manual_seed(SEED) / tf.random.set_seed(SEED)
```

Why this matters: Without a fixed seed, every run gives slightly different accuracy, so you can't tell whether "model B beat model A" was a real improvement or just luck from a different data split. Seeds turn noisy comparisons into fair ones.

Instructor insight: In interviews I ask "your teammate can't reproduce your 92% result — what do you check?" The strong answer walks through the four pillars: same data (hash), same code (git commit), same environment (pinned requirements / Docker), same seed. Weak candidates only mention the seed. If you can recite those four, you understand reproducibility.

Feature stores (conceptual)

As teams grow, everyone reinvents `days_since_last_order` slightly differently — and skew multiplies. A **feature store** solves this. Conceptually it is a central place that:

- **Computes features once** from raw data, on a schedule.
- **Serves them two ways:** an *offline store* (big historical tables for training) and an *online store* (fast key-value lookups for serving, e.g., "give me the latest features for user 123" in a few milliseconds).
- **Guarantees consistency** — the offline and online values come from the same definition, killing skew by design.
- Adds **feature reuse** (define once, many models use it) and **point-in-time correctness** (no accidentally training on future data — a subtle bug called *label leakage*).

Open-source example: **Feast** (free). You will not deploy a feature store in this 15-hour course, but you must be able to explain *what problem it solves*: consistent, reusable, skew-free features shared across training and serving.

```
+--> Offline store (history) --> Training
raw data --> [feature definitions]
          +--> Online store (latest) --> Serving API
          (same definition feeds both -> no skew)
```

Why this matters: In this course our "feature store" is just `features.py` imported by both sides. That's the same *principle* at small scale — one definition, two consumers. Understanding the principle is what matters for exams and interviews; the fancy tool is just an industrial version of the same idea.

Mini-FAQ

Q: Isn't storing dated data snapshots wasteful? A: Storage is cheap; a wrong or unreproducible model is expensive. For big data, DVC or Parquet compression keeps it manageable. Start with snapshots for small datasets — the habit matters more than the tool.

Q: I set the seed but still get different results on a friend's laptop. Why? A: Seeds only control randomness *within* a fixed environment. Different library versions, different OS/hardware (especially GPUs), or different data can still shift results. That's exactly why you also pin dependencies and version data — and ultimately containerize (note 06).

Q: Do we need a feature store for the capstone? A: No. A shared `features.py` used by both `train.py` and the FastAPI app gives you the core benefit (no skew) for free. Mention the feature store as the "grown-up" version if asked.

SESSION 3

03 — Experiment Tracking with MLflow

Learning objectives

By the end of this session you should be able to: - Explain *why* ad-hoc experiment tracking (notebook cells, filenames like `model_final_v2_REAL.pkl`) fails. - Use **MLflow Tracking** to log parameters, metrics, and artifacts, and compare runs. - Explain what a **run** and an **experiment** are in MLflow. - Describe the **Model Registry** and how model **versions** and **stages** support safe promotion to production.

Key terms

- **Experiment tracking** — systematically recording every training run's inputs and outputs.
- **Run** — one execution of training code; logs params, metrics, artifacts.
- **Experiment** — a named group of related runs (e.g., "churn-model").
- **Parameter** — an input you chose (learning rate, `n_estimators`).
- **Metric** — a measured output (accuracy, F1, RMSE), can be logged over time.
- **Artifact** — a file produced by a run (the model, a plot, a confusion matrix).
- **Model Registry** — a versioned catalog of models with lifecycle stages.
- **Stage** — a label on a model version: None / Staging / Production / Archived.

The problem: you can't improve what you don't remember

A typical student ML workflow: run a cell, note the accuracy on a sticky note, tweak a hyperparameter, run again. After 30 runs you have: - `model.pkl`, `model2.pkl`, `model_final.pkl`, `model_final_REAL.pkl` - no record of which hyperparameters made the best one - no way to reproduce run #14, which was actually your best

This is not a discipline problem — it's a *tooling* problem. Human memory and filenames don't scale to dozens of runs. **Experiment tracking** tools record, for every run: the code version, parameters, metrics, and output artifacts, automatically and searchably.

Instructor insight: The moment a team adopts experiment tracking is the moment ML stops being folklore ("I think the deeper tree was better?") and becomes engineering ("run a1b2 with max_depth=8 scored F1 0.81, here's the link"). In interviews, being able to say "we tracked every run in MLflow and promoted from the registry" signals you've worked on a real system.

MLflow, running locally and free

MLflow is an open-source platform. It has four parts; we use two: **Tracking** and **Model Registry**. It runs entirely on your laptop — no account, no cloud, no cost.

```
pip install mlflow==2.11.3

# Start the local tracking UI (defaults to a local SQLite/file store)
mlflow ui                # opens http://127.0.0.1:5000
```

By default MLflow writes to a local `./mlruns` folder. That's perfect for this course.

Logging a run

You wrap your training code in a `with mlflow.start_run()` block and call `log_param`, `log_metric`, `log_artifact`.

```
import mlflow
import mlflow.sklearn
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, f1_score

mlflow.set_experiment("churn-model")          # groups related runs

params = {"n_estimators": 200, "max_depth": 8, "random_state": 42}

with mlflow.start_run(run_name="rf-depth8"):
    model = RandomForestClassifier(**params).fit(X_tr, y_tr)
    preds = model.predict(X_te)

    mlflow.log_params(params)                  # inputs you chose
    mlflow.log_metric("accuracy", accuracy_score(y_te, preds)) # measured output
    mlflow.log_metric("f1", f1_score(y_te, preds))

    mlflow.log_artifact("data_manifest.json")  # WHICH data
    mlflow.sklearn.log_model(model, "model")   # the model itself
```

Why this matters: Every run is now a permanent, searchable record. Six months later you open the UI, sort by F1, click the best run, and see the exact parameters, the data manifest, and a downloadable model. No sticky notes. This single block replaces your entire "which file was best?" problem.

Comparing runs

Do a small hyperparameter sweep and MLflow lets you compare them side by side in the UI (or via the API).

```
for depth in [4, 8, 16, None]:
    with mlflow.start_run(run_name=f"rf-depth-{depth}"):
        m = RandomForestClassifier(n_estimators=200, max_depth=depth,
                                   random_state=42).fit(X_tr, y_tr)
        f1 = f1_score(y_te, m.predict(X_te))
        mlflow.log_param("max_depth", depth)
        mlflow.log_metric("f1", f1)
```

In the MLflow UI: open the `churn-model` experiment, tick several runs, click **Compare**. You get a table of params vs metrics and can plot metric-vs-parameter. This is how you see that F1 peaks at `max_depth=8` and overfitting hurts beyond it.

```
# You can also query runs programmatically
from mlflow.tracking import MlflowClient
client = MlflowClient()
runs = client.search_runs(experiment_ids=["1"], order_by=["metrics.f1 DESC"],
                           max_results=3)
for r in runs:
    print(r.info.run_id, r.data.params.get("max_depth"), r.data.metrics["f1"])
```

Watch out — log the *data version* too. Students log params and metrics but forget the artifact that says *which dataset* was used. Then two runs with identical params show different F1 and it looks like a bug — but the data changed underneath. Always log the data manifest/hash (note 02) as an artifact so a run is fully reproducible.

Metrics over time (for iterative training)

For models trained in steps (neural nets, boosting), log a metric with a `step` so MLflow draws a learning curve.

```
for epoch in range(1, 21):
    train_one_epoch(...)
    val_loss = evaluate(...)
    mlflow.log_metric("val_loss", val_loss, step=epoch)  # plotted as a curve
```

Why this matters: A learning curve tells you *why* a run is bad — overfitting (val loss rises while train loss falls), underfitting (both high), or needs more epochs. A single final number hides all of that.

The Model Registry

Tracking answers "what did I try?" The **Model Registry** answers "which model is *the* model right now, and is it safe to deploy?" It is a versioned catalog:

- You **register** a model from a good run under a name, e.g., `churn-model`.
- Each registration creates a new **version**: v1, v2, v3...
- Each version carries a **stage**: `None`, `Staging`, `Production`, `Archived`.
- Your serving code loads *by stage*, e.g., "give me the current Production model," not by a hard-coded file path.

```
# Register the model produced by a run
result = mlflow.register_model(
    model_uri="runs:/<RUN_ID>/model",
    name="churn-model"
)  # -> creates version N

# Promote it after it passes validation
from mlflow.tracking import MlflowClient
client = MlflowClient()
client.transition_model_version_stage(
    name="churn-model", version=result.version, stage="Staging"
)

# ...run integration tests, then:
client.transition_model_version_stage(
    name="churn-model", version=result.version, stage="Production",
    archive_existing_versions=True  # old Production -> Archived
)
```

```
# Serving code loads by STAGE, never by filename
model = mlflow.pyfunc.load_model("models:/churn-model/Production")
```

Why this matters: Loading by stage decouples deployment from training. To ship a better model you just transition v4 to Production — no code change, no redeploy of a hard-coded path. To roll back, transition v3 back to Production. This is the safe-promotion pattern real teams use, and it's the bridge to CI/CD in note 07.

Instructor insight: The registry gives you an *audit trail*: who promoted which version, when, and from which run/data. In regulated settings (finance, healthcare — very relevant to India's growing AI sector) that lineage isn't optional. Even for a class project, demonstrating "run -> register -> Staging -> Production -> rollback" in the MLflow UI is a strong capstone moment.

A clean end-to-end tracking flow

```
train.py runs a sweep
-> each run logged (params, metrics, data manifest, model artifact)
-> pick best run in the UI
-> mlflow.register_model -> churn-model v3
-> transition v3 to Staging -> run validation tests
-> transition v3 to Production (v2 auto-archived)
-> FastAPI loads models:/churn-model/Production
```

Mini-FAQ

Q: MLflow vs. just writing to a CSV log? A: A CSV can hold metrics, but MLflow also stores the *model artifact*, the code version, comparison UI, and the registry with stages. The registry — load-by-stage and rollback — is the part a CSV can never give you.

Q: Do we need a server to run MLflow? A: No. `mlflow ui` on your laptop with the default local store is enough for this course. A shared server matters only when a team needs one source of truth. Same API either way.

Q: What's the difference between an experiment and a run? A: A run is one training execution. An experiment is a named folder that groups related runs (e.g., all attempts at the churn model), so you can compare them. One experiment, many runs.

SESSION 4

04 — Model Serialization & Packaging

Learning objectives

By the end of this session you should be able to: - Serialize a trained model with **pickle**, **joblib**, and understand when **ONNX** is the better choice. - List the trade-offs (portability, speed, size, security) between these formats. - Explain why a serialized model alone is not enough, and what else must travel with it. - Diagnose and prevent the "**works on my machine**" problem through dependency pinning and packaging.

Key terms

- **Serialization** — converting an in-memory object (your trained model) into bytes on disk.
- **Deserialization / loading** — the reverse: bytes back into a usable object.
- **pickle** — Python's built-in object serializer.
- **joblib** — a serializer optimized for large NumPy arrays (common in scikit-learn).
- **ONNX** — Open Neural Network Exchange: a language-agnostic model format.
- **Artifact bundle** — the model file *plus* everything needed to load and use it correctly.

Why serialization matters

You trained a model. It lives in your laptop's RAM as a Python object. The moment your script ends, it's gone. To serve it tomorrow — or on a different machine, in a Docker container, behind an API — you must **serialize** it: freeze it to disk now, thaw it later.

The catch: a serialized model is only useful if you can load it back into a *compatible* environment and feed it *identical* inputs. Get either wrong and you have a file that either won't open or produces wrong answers.

pickle — the default

pickle is Python's built-in. It can serialize almost any Python object.

```
import pickle

# Save
with open("model.pkl", "wb") as f:
    pickle.dump(model, f)

# Load
with open("model.pkl", "rb") as f:
    model = pickle.load(f)
```

Pros: built-in, zero dependencies, handles arbitrary Python objects. **Cons:** - **Version-fragile.** A pickle saved with scikit-learn 1.4 may fail or misbehave when loaded with 1.2 or 1.6. - **Python-only.** Can't load it in a Java or C++ service. - **Insecure.** Unpickling *executes code*. Never `pickle.load` a file from an untrusted source — it can run arbitrary commands on your machine.

Watch out — the security foot-gun. `pickle.load()` on a malicious file can delete your files or exfiltrate data, because unpickling can invoke arbitrary code. Rule: only load pickles you (or your CI) created. Never accept a `.pkl` upload from users and load it. This is a real interview question.

joblib — better for scikit-learn

joblib uses the same pickle protocol under the hood but is optimized for objects containing large NumPy arrays (most sklearn models). It's the format scikit-learn's own docs recommend.

```
import joblib

joblib.dump(model, "model.joblib")          # often smaller & faster for big arrays
model = joblib.load("model.joblib")
```

Pros: faster and smaller than pickle for array-heavy models; simple API. **Cons:** same version-fragility and Python-only, insecure-load caveats as pickle.

Why this matters: For a scikit-learn model in this course, **joblib** is your default. It's the sweet spot: minimal effort, good performance, and it's what the ecosystem expects. Reach for anything fancier only when you have a specific reason.

ONNX — portability across languages and runtimes

ONNX is a *format*, not a Python object dump. You convert your model into a standard computation graph that any ONNX runtime (Python, C++, C#, JavaScript, mobile) can execute. It decouples the model from the training library entirely.

```
# Convert an sklearn model to ONNX
from skl2onnx import to_onnx
import numpy as np

onx = to_onnx(model, X_tr[:1].astype(np.float32))
with open("model.onnx", "wb") as f:
    f.write(onx.SerializeToString())
```

```
# Run it with onnxruntime -- no scikit-learn needed at serving time!
import onnxruntime as ort
sess = ort.InferenceSession("model.onnx")
outputs = sess.run(None, {"X": X_te.astype(np.float32)})
```

Pros: language/runtime-independent; often faster inference; no training library at serve time; not the pickle security risk. **Cons:** conversion step can be fiddly; not every custom Python transformer converts cleanly; adds complexity.

Why this matters: Choose ONNX when you must serve in a non-Python environment (mobile, a C++ backend, the browser) or need maximum inference speed. For a pure-Python FastAPI service in this course, joblib is usually enough — but you should be able to *explain* when and why ONNX wins.

Format decision cheat-sheet

NEED	BEST CHOICE
Quick sklearn model, Python-only service	joblib
Arbitrary Python object, tiny/simple	pickle
Cross-language / mobile / max speed	ONNX

NEED	BEST CHOICE
Deep learning, staying in framework	native (<code>torch.save</code> , <code>SavedModel</code>)

A model file is NOT enough

The most common packaging mistake: shipping only `model.joblib`. To *correctly* load and use a model, five things must travel together:

1. **The model file** (`model.joblib`).
2. **The exact library versions** (`requirements.txt` , pinned) — a model saved with sklearn 1.4.2 must be loaded with 1.4.2.
3. **The feature/preprocessing code** (`features.py`) — same transforms as training (note 02).
4. **The input schema** — column names, order, dtypes the model expects.
5. **Metadata** — training data version/hash, metrics, date, git commit.

```
# A self-describing artifact bundle
import joblib, json

bundle = {
    "model": model,
    "feature_names": ["income_k", "age", "orders_per_year"],
    "sklearn_version": "1.4.2",
    "data_sha256": "9f2c...",
    "trained": "2026-07-17",
    "metrics": {"f1": 0.81},
}
joblib.dump(bundle, "churn_bundle.joblib")
# At load time you can VALIDATE the environment matches:
import sklearn
b = joblib.load("churn_bundle.joblib")
assert sklearn.__version__ == b["sklearn_version"], "sklearn version mismatch!"
```

Why this matters: This bundle makes the model *self-describing*. Whoever loads it can check the sklearn version, confirm the feature order, and see which data trained it — before serving a single prediction. Note that MLflow's `log_model` (note 03) does much of this bundling for you automatically, which is why we prefer the registry.

The "works on my machine" problem

Classic scenario: your model runs perfectly on your laptop, crashes on your teammate's, and produces *different numbers* on the server. Causes, in order of frequency:

1. **Different library versions.** You have numpy 1.26, the server has 1.24. Solution: **pin** everything (note 02) and `pip install -r requirements.txt` everywhere.
2. **Different Python version.** 3.11 vs 3.9. Pickles and some libs are sensitive to this. Pin the Python version too.
3. **Missing system libraries** (e.g., `libgomp` for some models). Solution: containerize (note 06).
4. **Different data/features** at serve time. Solution: shared `features.py` + schema validation.
5. **OS / hardware differences** (float behavior across CPUs/GPUs).

```
# The reproducible-install habit
python --version          # record it (e.g., 3.11.x)
pip freeze > requirements.txt # pin everything
pip install -r requirements.txt # identical install anywhere
```

Instructor insight: "Works on my machine" is not a joke — it's a signal that your artifact is under-specified. The industry's answer evolved in three steps: pin dependencies -> bundle metadata -> put the *entire environment* in a Docker image (note 06). Each step removes one more source of drift. When someone says a model "doesn't work in prod," 80% of the time it's an unpinned dependency or a feature mismatch. Package the environment, not just the model.

Watch out: Don't commit large `.joblib` / `.pkl` / `.onnx` files straight into Git — repos bloat and slow to a crawl. Store models as MLflow artifacts or use DVC/Git-LFS, and commit only pointers. Git is for code; models are data.

Mini-FAQ

Q: Can I load a pickle made with a newer sklearn on an older one? A: Usually no, and even "successful" loads can behave subtly wrong. Always load with the *same* version you saved with — which is exactly why you pin dependencies and travel them with the model.

Q: If ONNX is faster and safer, why not always use it? A: Conversion adds friction and doesn't support every custom Python step cleanly. For a Python-only service where joblib works, the extra complexity isn't worth it. Use ONNX when you have a concrete need: cross-language serving, mobile, or a hard latency target.

Q: How does MLflow relate to all this? A: `mlflow.sklearn.log_model` serializes the model *and* records the environment (a conda/pip spec) and input schema for you, then the registry versions it. It's the "bundle everything" idea, automated. Prefer it over hand-rolling joblib bundles once you're using MLflow.

SESSION 5

05 — Serving Models with FastAPI

Learning objectives

By the end of this session you should be able to: - Distinguish **batch** from **online** (real-time) serving and pick the right one for a use case. - Build a minimal **FastAPI** REST service that loads a model and returns predictions. - Define **request/response schemas** and validate inputs with **Pydantic**. - Add **health** and **inference** endpoints and reason about basic **latency**.

Key terms

- **Serving** — making a trained model answer prediction requests.
- **Batch inference** — scoring many records at once, offline, on a schedule.
- **Online inference** — scoring one (or a few) records live, on demand, low latency.
- **REST API** — HTTP endpoints that accept requests and return responses (usually JSON).
- **FastAPI** — a modern, fast Python web framework with built-in validation and docs.
- **Pydantic** — the library FastAPI uses to define and validate data schemas.
- **Endpoint** — a specific URL + HTTP method your service responds to.
- **Latency** — time from request sent to response received.

Batch vs. online serving

Before writing any code, decide *how* predictions are consumed:

- **Batch:** run predictions for a large set on a schedule and store the results. Example: every night, score all customers for churn risk and write to a table the marketing team queries in the morning. High throughput, latency doesn't matter (minutes/hours are fine), simple to run (a cron job + a script).
- **Online:** predict on demand, one request at a time, while a user waits. Example: a fraud check on a card swipe, or a recommendation as a page loads. Needs low latency (tens to low hundreds of milliseconds) and an always-on service.

```
Batch:    [scheduler] -> load model -> score 1M rows -> write to DB    (latency irrelevant)
Online:   user/app -> HTTP request -> API scores 1 row -> HTTP response (latency critical)
```

Why this matters: Teams waste weeks building a real-time API when a nightly batch job would do — and vice versa. Ask: "does someone wait for this answer *right now*?" If no, batch (cheaper, simpler). If yes, online. This course builds an online API because it's the harder, more instructive case.

FastAPI: a minimal serving service

FastAPI is the standard modern choice: fast, async-capable, and it auto-generates interactive API docs and input validation for free. We serve the model behind an HTTP endpoint.

```
# app.py
import joblib
import pandas as pd
from fastapi import FastAPI
from pydantic import BaseModel, Field
from features import build_features # SAME feature code as training (note 02)

app = FastAPI(title="Churn Model API")

# Load the model ONCE at startup, not per request (see latency section)
model = joblib.load("model.joblib")

@app.get("/health")
def health():
    return {"status": "ok"}

class CustomerRequest(BaseModel):
    income: float = Field(gt=0, description="Annual income")
    birth_year: int = Field(ge=1900, le=2026)
    total_orders: int = Field(ge=0)
    tenure_years: float = Field(gt=0)

class PredictionResponse(BaseModel):
    churn_probability: float
    will_churn: bool

@app.post("/predict", response_model=PredictionResponse)
def predict(req: CustomerRequest):
    raw = pd.DataFrame([req.dict()])
    X = build_features(raw) # no train/serve skew
    prob = float(model.predict_proba(X)[0, 1])
    return PredictionResponse(churn_probability=prob, will_churn=prob > 0.5)

# Run it with uvicorn (an ASGI server)
uvicorn app:app --host 0.0.0.0 --port 8000 --reload
# Interactive docs auto-generated at http://localhost:8000/docs
```

Why this matters: In ~30 lines you have a production-shaped service: typed inputs, validated automatically, model loaded once, shared feature code, and self-documenting endpoints at [/docs](#). That [/docs](#) page (Swagger UI) lets your instructor and classmates try the model in a browser with zero extra work.

Request/response schemas with Pydantic

The `BaseModel` classes above are **Pydantic schemas**. They are your API's contract. FastAPI uses them to: - **Parse** incoming JSON into typed Python objects. - **Validate** it automatically — wrong types or out-of-range values get rejected with a clear 422 error *before* your model ever runs. - **Document** the API (the [/docs](#) page is generated from these schemas).

```
// A valid request body to POST /predict
{ "income": 850000, "birth_year": 1995, "total_orders": 12, "tenure_years": 3.5 }
```

```
// If income is negative, FastAPI auto-rejects with 422 -- your code never runs:
{ "detail": [ { "loc": ["body", "income"], "msg": "Input should be greater than 0" } ] }
```

Why this matters: Validation is your first line of defense against garbage input and a subtle guard against skew. If someone sends `birth_year: 2050`, you reject it loudly instead of computing a negative age and returning a confident wrong prediction. Never trust the caller; let Pydantic enforce the contract.

Watch out — validate ranges, not just types. Students declare `income: float` and stop. But `income: -5` is a valid float and a nonsense input. Use `Field(gt=0, le=...)` to encode the *domain*. Type-correct but domain-invalid inputs are a classic source of silent bad predictions.

Health and inference endpoints

Two categories of endpoint every serving service needs:

- **Health / liveness** (`GET /health`) — returns 200 if the service is up. Used by the host/load balancer to know whether to send traffic or restart the container. It should be cheap and *not* run the model.
- **Inference** (`POST /predict`) — the actual prediction. Often also a `/predict/batch` that accepts a list.

```
from typing import List

@app.post("/predict/batch")
def predict_batch(reqs: List[CustomerRequest]):
    raw = pd.DataFrame([r.dict() for r in reqs])
    X = build_features(raw)
    probs = model.predict_proba(X[:, 1])
    return [{"churn_probability": float(p), "will_churn": bool(p > 0.5)} for p in probs]
```

Why this matters: Hosting platforms (Render, HF Spaces, Kubernetes) *ping your health endpoint* to decide if the service is alive. No health endpoint -> the platform can't tell a hung service from a healthy one -> it either sends traffic to a dead app or never marks it ready. It's a five-line function that makes your deployment operable. A batch endpoint also amortizes overhead when a caller has many rows.

Latency basics

Latency is the time from request to response. For online serving it's a first-class metric. Where it goes:

1. **Network** — request/response travel time (you don't control much of this).
2. **Deserialization/validation** — parsing JSON, Pydantic checks (small).
3. **Feature building** — your `build_features` code.
4. **Model inference** — `predict` / `predict_proba`.
5. **Serialization** — building the JSON response.

Practical rules: - **Load the model once at startup**, not inside the endpoint. Loading a joblib file per request can add tens to hundreds of ms — the #1 beginner latency bug. - **Measure percentiles, not averages.** p50 (median) hides pain; p95/p99 (the slowest 5%/1%) is what users actually complain about. One slow-in-20 request is very visible. - **Batch when you can.** Scoring 100 rows in one `predict` call is far cheaper than 100 separate calls.

```
import time
from fastapi import Request

@app.middleware("http")
async def add_timing(request: Request, call_next):
    start = time.perf_counter()
    response = await call_next(request)
    ms = (time.perf_counter() - start) * 1000
    response.headers["X-Process-Time-ms"] = f"{ms:.1f}"
    return response
```

Why this matters: This middleware stamps every response with how long it took — the cheapest possible latency monitoring, and it feeds the drift/monitoring work in note 09. You cannot improve latency you don't measure.

Instructor insight: In a real interview I'll ask "your model API's p99 latency spiked from 50ms to 800ms — where do you look?" Strong answers walk the list: is the model being reloaded per request? Did feature code start hitting a slow database? Did input size grow? Is the host cold-starting (note 08)? The candidate who separates *model time* from *everything-around-the-model time* stands out — because it's almost always the "everything around" part.

Mini-FAQ

Q: Flask or FastAPI? A: FastAPI for new ML services. You get automatic validation, async support, and auto-generated docs, which Flask needs extra libraries for. Flask is fine and simpler, but FastAPI's built-in Pydantic validation is exactly what model serving needs.

Q: Why load the model at startup instead of per request? A: Loading is slow (disk read + deserialize) and the model doesn't change between requests. Load once into a module-level variable; every request reuses it. Loading per request can make you 10-100x slower for no benefit.

Q: Is one FastAPI process enough for many users? A: For a class project, yes. For real traffic you run multiple worker processes (e.g., `uvicorn --workers 4` or gunicorn) and multiple container replicas behind a load balancer (note 08). Start simple; scale when you measure a need.

SESSION 6

06 — Containerization with Docker

Learning objectives

By the end of this session you should be able to: - Explain the difference between **containers** and **virtual machines** and why containers won for ML deployment. - Read and write a **Dockerfile** for a FastAPI model service. - Explain **images, layers, tags, and ports**, and why layer caching speeds up builds. - Use `docker build` and `docker run`, and explain how containers finally kill "works on my machine."

Key terms

- **Container** — a lightweight, isolated process bundling your app + its dependencies, sharing the host OS kernel.
- **Image** — the read-only blueprint a container is created from.
- **Layer** — one step in an image, cached and reused across builds.
- **Tag** — a label on an image version, e.g., `churn-api:1.0`.
- **Dockerfile** — the recipe that defines how to build an image.
- **Port mapping** — connecting a container's internal port to a host port.
- **Registry** — a place to store/share images (Docker Hub, GHCR).

Containers vs. virtual machines

Both isolate software, but at different levels:

- A **virtual machine (VM)** virtualizes *hardware*. Each VM runs a full guest operating system on top of a hypervisor. Heavy: gigabytes of disk, minutes to boot, significant RAM overhead — because you're running a whole OS.
- A **container** virtualizes the *operating system*. Containers share the host's kernel and isolate only the process, filesystem, and network. Light: tens/hundreds of MB, starts in seconds.

VM:	App + libs + FULL guest OS	->	Hypervisor	->	Host OS	->	Hardware
Container:	App + libs	->	Docker	->	Host OS	->	Hardware
	(shares the host kernel; no guest OS per container)						

Why this matters: For ML serving you want to package *your exact environment* (Python 3.11, numpy 1.26.4, your model) and run many identical copies cheaply. A VM per model is wasteful; a container per model is cheap and fast. That efficiency is why every free host we use (Render, HF Spaces) runs containers, not VMs.

Why containers give reproducibility

Recall the "works on my machine" problem (note 04): different Python versions, missing system libraries, unpinned packages. A container fixes this by **shipping the entire environment**, not just the code:

- the exact OS base (e.g., `python:3.11-slim`),
- the exact system libraries,
- the exact pinned Python packages,

- your code and model.

If it runs in the container on your laptop, it runs *identically* in the container on the server — because it's the same environment, byte for byte. The container **is** the reproducibility guarantee that pinning alone can't fully provide.

Instructor insight: Docker is where all the earlier habits pay off. Pinned `requirements.txt` (note 02) becomes a reproducible image. The self-describing model bundle (note 04) rides inside it. When I say "package the environment, not just the model," the Dockerfile is literally how you do it. A model in a container is the unit teams actually deploy.

Anatomy of a Dockerfile

A **Dockerfile** is an ordered recipe. Each instruction creates a **layer**.

```
# 1. Base image: a minimal Debian with Python 3.11 preinstalled
FROM python:3.11-slim

# 2. Set a working directory inside the container
WORKDIR /app

# 3. Copy ONLY requirements first (layer-caching trick, explained below)
COPY requirements.txt .

# 4. Install pinned dependencies
RUN pip install --no-cache-dir -r requirements.txt

# 5. Copy the rest of the app (code + model artifact)
COPY . .

# 6. Document the port the app listens on
EXPOSE 8000

# 7. The command that runs when the container starts
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

Instruction by instruction: - **FROM** — the starting point. - `-slim` variants are smaller (fewer preinstalled packages) — smaller images deploy faster and have less to patch. - **WORKDIR** — sets the current directory for later commands. - **COPY** — copies files from your machine into the image. - **RUN** — executes a command at *build time* (here, installing packages). - **EXPOSE** — documentation of the listening port (doesn't itself publish it). - **CMD** — the default command run *when the container starts*. Note `--host 0.0.0.0` so the app is reachable from outside the container, not just its own localhost.

Watch out — `--host 0.0.0.0` matters. If uvicorn binds to `127.0.0.1` inside the container, it only listens to the container's *own* loopback, and no outside request can reach it. In a container you almost always bind `0.0.0.0`. This trips up nearly everyone the first time; the container "runs" but nothing can connect.

Images, layers, tags, and ports

Layers and caching. Each Dockerfile instruction produces a cached layer. On rebuild, Docker reuses cached layers for instructions whose inputs didn't change. That's *why we copy `requirements.txt` before the rest of the code*:

```
COPY requirements.txt .    <- changes rarely
RUN pip install ...        <- SLOW; cached & skipped when requirements unchanged
COPY . .                  <- changes often (your code)
```

If you copied everything first, editing one line of `app.py` would invalidate the cache and force a full, slow reinstall every build. Ordering from least- to most-frequently-changed keeps builds fast.

Why this matters: A cold build might take 3 minutes (installing packages); a cached rebuild after a code tweak takes 3 seconds. This ordering trick is the single biggest build-speed lever, and it matters a lot when CI/CD (note 07) rebuilds on every push.

Tags. A tag names an image version: `churn-api:1.0`, `churn-api:latest`. Tag deliberately — `latest` is ambiguous, so use real versions for anything you deploy or roll back.

Ports. A container has its own network namespace. To reach a container's port 8000 from your machine you map it: `-p 8000:8000` means "host port 8000 -> container port 8000."

Build and run

```
# Build an image from the Dockerfile in the current directory, tag it
docker build -t churn-api:1.0 .

# Run a container from that image, mapping port 8000, in the background
docker run -d -p 8000:8000 --name churn churn-api:1.0

# Check it
curl http://localhost:8000/health      # -> {"status":"ok"}

# See running containers, view logs, stop it
docker ps
docker logs churn
docker stop churn && docker rm churn
```

Passing configuration at run time (never bake secrets into the image — note 08):

```
docker run -d -p 8000:8000 \
  -e MODEL_STAGE=Production \
  -e LOG_LEVEL=info \
  churn-api:1.0
```

Why this matters: `docker build` once, then `docker run` the *same image* on your laptop, a teammate's machine, and the free host — with identical behavior. The image is the deployable artifact. When you deploy to Render or HF Spaces in note 08, you're just running this same image on their infrastructure.

A `.dockerignore` keeps images lean:

```
# .dockerignore
__pycache__/
*.pyc
.git/
.venv/
mlruns/
notebooks/
```

Watch out — don't ship the whole repo. Without a `.dockerignore`, `COPY . .` drags in your `.git` history, `virtualenv`, `datasets`, and `notebooks` — bloating the image and sometimes leaking secrets. Ignore everything the running service doesn't need. Also: never `COPY` a `.env` file with credentials into the image; pass secrets at runtime.

Putting it together

```
Dockerfile + requirements.txt + app.py + features.py + model.joblib
|
v  docker build -t churn-api:1.0 .
[ image: churn-api:1.0 ]  <- fully reproducible environment
|
v  docker run -p 8000:8000 churn-api:1.0
[ running container ] -> POST /predict -> prediction
|
v  (push to a registry, then the free host runs the same image)
```

Mini-FAQ

Q: Do I need Docker Desktop, and is it free? A: Docker Engine is free and open-source. Docker Desktop is free for students and small use. On Linux you can use the engine directly. For this course, local Docker is enough — no paid tier needed.

Q: My container builds but the app is unreachable at localhost:8000. Why? A: Two usual causes: (1) you didn't map the port (`-p 8000:8000`), or (2) `uvicorn` is bound to `127.0.0.1` instead of `0.0.0.0` inside the container. Fix both and it'll connect.

Q: Image is huge (1GB+). How do I shrink it? A: Use a `-slim` base, add a `.dockerignore`, use `pip install --no-cache-dir`, and don't copy `datasets/notebooks` into the image. For advanced cases, multi-stage builds separate build tools from the runtime. Smaller images build, push, and cold-start faster.

SESSION 7

07 — CI/CD for Machine Learning

Learning objectives

By the end of this session you should be able to: - Define **CI** and **CD** and explain what a pipeline automates. - Write a **GitHub Actions** workflow that lints, tests, and builds on every push. - Design **ML-specific tests**: data tests, behavioral tests, and accuracy thresholds. - Explain how CI/CD for ML differs from CI/CD for ordinary software.

Key terms

- **CI (Continuous Integration)** — automatically build and test every code change.
- **CD (Continuous Delivery/Deployment)** — automatically release changes that pass CI.
- **Pipeline / workflow** — the automated sequence of steps that runs on a trigger.
- **GitHub Actions** — GitHub's built-in CI/CD system (free for public repos).
- **Runner** — the machine that executes a workflow.
- **Data test** — a check on the data's shape/quality.
- **Behavioral test** — a check on how the model behaves on crafted inputs.
- **Accuracy threshold / gate** — a minimum metric a model must beat to be released.

What CI/CD is, in one paragraph

Every time you push code, a robot should automatically check it doesn't break anything and, if it passes, ship it. **CI** is the "check it" half: install, lint, run tests, build the Docker image. **CD** is the "ship it" half: deploy the passing build to the host. The value is *fast, trustworthy feedback* — you learn within minutes that a change is safe (or not), instead of finding out from users in production. It also removes human error from releases: the same steps run the same way every time.

```
git push -> [ CI: lint -> unit tests -> ML tests -> docker build ] -> [ CD: deploy ]
              any step fails -> stop, notify, DON'T deploy
```

GitHub Actions basics (free)

A workflow is a YAML file in `.github/workflows/`. GitHub Actions is **free for public repositories** — ideal for a student project. Structure: a workflow has **triggers** (`on:`), one or more **jobs**, and each job has **steps**.

```
# .github/workflows/ci.yml
name: CI

on:
  push:
    branches: [ main ]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest          # a free Linux runner
    steps:
      - uses: actions/checkout@v4    # get the code

      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      - name: Install pinned dependencies
        run: pip install -r requirements.txt

      - name: Lint
        run: |
          pip install ruff
          ruff check .

      - name: Run tests
        run: pytest -q
```

Why this matters: This file, committed once, means every push and every pull request is automatically linted and tested on a clean machine — catching the "works on my machine" bug (note 04) before merge, for free. If it passes on a fresh Ubuntu runner with only your pinned `requirements.txt`, your environment really is reproducible.

Adding a build step (and, on `main`, deploy):

```
build-and-deploy:
  needs: test                      # only runs if 'test' job passed
  if: github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - name: Build Docker image
      run: docker build -t churn-api:${{ github.sha }} .
    # Deploy step: e.g., trigger a Render deploy hook, or push to a registry.
    - name: Trigger deploy
      run: curl -X POST "${{ secrets.RENDER_DEPLOY_HOOK }}"
```

Watch out — never hard-code secrets in the YAML. Deploy hooks, API keys, and tokens go in **GitHub repository Secrets** and are referenced as `${{ secrets.NAME }}`. A secret committed to a public repo is compromised the instant you push and must be rotated. This is the most common — and most costly — beginner CI/CD mistake.

ML-specific tests

Ordinary software tests check code logic. ML systems can pass every code test and still ship a terrible model, because **the model's behavior comes from data, not code**. So you add three extra layers of testing:

1. Data tests

Validate the *data* before training or serving. Catches upstream breakage (note 01's data dependencies).

```
def test_data_schema(df):
    assert list(df.columns) == ["income", "birth_year", "total_orders", "tenure_years",
                                "churned"]
    assert df["income"].min() > 0                # no negative incomes
    assert df["birth_year"].between(1900, 2026).all()
    assert df["churned"].isin([0, 1]).all()      # label is binary
    assert df.isnull().mean().max() < 0.05      # <5% missing per column
```

2. Behavioral tests

Check the *model's behavior* on crafted inputs — like unit tests for predictions. Inspired by the "CheckList" idea: invariance, directional, and minimum-functionality tests.

```
def test_higher_income_lowers_churn(model):
    # Directional test: raising income should not INCREASE churn probability
    low = build_features(pd.DataFrame([{"income":200000,"birth_year":1990,
                                         "total_orders":5,"tenure_years":2}]))
    high = build_features(pd.DataFrame([{"income":2000000,"birth_year":1990,
                                         "total_orders":5,"tenure_years":2}]))
    assert model.predict_proba(high)[0,1] <= model.predict_proba(low)[0,1] + 0.05

def test_output_is_valid_probability(model):
    # Minimum functionality: output must be in [0,1]
    p = model.predict_proba(sample_features)[0,1]
    assert 0.0 <= p <= 1.0
```

3. Accuracy threshold (the release gate)

The model must beat a minimum metric on a held-out set, or the pipeline **fails and refuses to deploy**.

```
def test_model_meets_threshold(model, X_test, y_test):
    from sklearn.metrics import f1_score
    f1 = f1_score(y_test, model.predict(X_test))
    MIN_F1 = 0.75
    assert f1 >= MIN_F1, f"Model F1 {f1:.3f} below gate {MIN_F1}"
```

Why this matters: These gates turn "I think the model is fine" into an automatic, enforced contract. A retrain on bad new data that drops F1 to 0.60 will *fail the gate and never reach users*. That's the single most important safety mechanism in ML CI/CD — it protects production from a bad model the same way unit tests protect it from a bad code change.

Instructor insight: A subtle but powerful gate is comparing the *candidate* model against the *current Production* model (from the MLflow registry, note 03), not just a fixed threshold. Deploy the new model only if it's at least as good as what's live. This prevents silent regressions where each retrain is "fine" against a static bar but quietly worse than last month. Interviewers love when you mention champion/challenger comparison.

How ML CI/CD differs from normal CI/CD

ASPECT	NORMAL SOFTWARE	ML SYSTEMS
What's tested	code logic	code + data + model behavior
Determinism	same input -> same output	training has randomness; results vary
"Passing" means	tests green	tests green and metric gate cleared
What you version	code	code + data + model
What can silently break it	a code bug	drift , bad upstream data, skew
Extra CD concept	deploy code	may retrain (Continuous Training)

Two ideas unique to ML: - **Continuous Training (CT)**. Beyond CI/CD, ML pipelines may *automatically retrain* on fresh data (triggered on a schedule or by drift alerts — note 09), then run the same test gates before promotion. - **Testing is probabilistic**. You can't assert `prediction == 0.83`. You assert *properties* (in [0,1], directionally correct) and *aggregate metrics above a threshold*, not exact values.

Watch out — flaky ML tests. If you don't fix random seeds (note 02), your accuracy-gate test will pass on some runs and fail on others for no real reason. A flaky gate gets ignored, and an ignored gate is useless. Seed everything, and give thresholds a small, honest tolerance rather than an impossibly tight one.

A realistic ML CI/CD flow

```
Developer pushes feature code
-> CI: lint + unit tests + DATA tests + BEHAVIORAL tests          (fast, always)
-> (optional) retrain on latest data version
-> ACCURACY GATE: candidate F1 >= threshold AND >= current Prod  (refuse if not)
-> build Docker image, tag with git SHA
-> register/ promote model in MLflow registry to Staging
-> CD: deploy image to free host (Render / HF Spaces)
-> smoke test /health and a known /predict in production
```

Mini-FAQ

Q: Is GitHub Actions really free? A: Yes, unlimited minutes for **public** repositories, and a monthly free allowance for private ones. For an open student project it's completely free — which is why we use it over paid CI services.

Q: Should CI retrain the model every push? A: Usually no. Retraining is slow and expensive, and code changes don't require it. Separate the two: run tests on every push; retrain on a schedule or a drift trigger (note 09), then run the gates. Keep fast checks fast.

Q: What's the smallest useful ML CI I can build for the capstone? A: One `ci.yml` that installs pinned deps, runs `pytest` (including a schema data test and an F1-threshold test), and builds the Docker image. That demonstrates the whole idea — code test + data test + model gate + build — in one file.

SESSION 8

08 — Deployment to a Free Host

Learning objectives

By the end of this session you should be able to: - Compare **self-managed** deployment vs. **managed endpoints** and the trade-offs. - Deploy a containerized model service to a **free host** (Hugging Face Spaces or Render). - Explain **endpoints, scaling basics, and cold starts** and how they affect users. - Handle **environment configuration and secrets** correctly (never in the image).

Key terms

- **Deployment** — making your service reachable by real callers over the network.
- **Self-managed** — you run and maintain the server/VM/cluster yourself.
- **Managed endpoint** — a platform runs your container and gives you a URL.
- **Endpoint** — the public URL your API answers on.
- **Scaling** — adjusting capacity (replicas) to match load.
- **Cold start** — the delay when a stopped/idle service must spin up before serving.
- **Environment variable** — a runtime config value passed into the container.
- **Secret** — a sensitive value (API key, token) that must not be committed.

Self-managed vs. managed

Two ends of a spectrum for running your container:

- **Self-managed** — you rent a raw VM (or run your own box/cluster), install Docker, run the container, and are responsible for OS patches, restarts on crash, scaling, TLS certificates, and uptime. Maximum control and flexibility; maximum operational burden. This is the SRE's world (note 01).
- **Managed endpoint / PaaS** — you hand the platform a container image (or a Git repo with a Dockerfile) and it runs it, gives you an HTTPS URL, restarts it on crash, and often auto-scales. Less control, far less operational work. Examples we use for free: **Hugging Face Spaces** and **Render**.

Self-managed: you own [OS patching | restarts | scaling | TLS | monitoring | the app]
 Managed: platform owns [OS | restarts | scaling | TLS] ; you own [the app]

Why this matters: For a 15-hour course and for most small teams, managed is the right call — you spend your time on the model and API, not on babysitting a server. The industry lesson: pay (or trade control) to avoid undifferentiated ops work unless running it yourself is your actual value-add. Free managed hosts give students production-shaped experience at zero cost.

Deploying a container to a free host

Because we containerized in note 06, deployment is mostly "point the host at the image/repo." Two free paths:

Path A — Hugging Face Spaces (Docker SDK)

Create a Space, choose the **Docker** SDK, and push a repo containing your **Dockerfile**. HF builds the image and serves it at `https://<user>-<space>.hf.space`. Note: Spaces expect the app on **port 7860** by default.


```
# Minimal change for HF Spaces: listen on 7860
EXPOSE 7860
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "7860"]
```

Path B — Render (free web service)

Connect your GitHub repo; Render detects the `Dockerfile`, builds, and deploys on every push to `main` (that's your CD from note 07). It provides an HTTPS URL and uses your **health endpoint** to decide when the service is ready.

```
# render.yaml (optional infra-as-code)
services:
- type: web
  name: churn-api
  env: docker
  healthCheckPath: /health
  envVars:
    - key: MODEL_STAGE
      value: Production
```

```
# After deploy, verify from anywhere:
curl https://churn-api.onrender.com/health # {"status":"ok"}
curl -X POST https://churn-api.onrender.com/predict \
  -H "Content-Type: application/json" \
  -d '{"income":850000,"birth_year":1995,"total_orders":12,"tenure_years":3.5}'
```

Why this matters: The *same Docker image* you tested on your laptop now serves the public internet over HTTPS, for free, redeploying automatically on every push. That is a complete, real deployment story — the thing most bootcamps only talk about. Your capstone can end with a live, shareable URL.

Watch out — the port must match. Each host expects a specific port (HF Spaces: 7860; Render: it injects a `$PORT` env var). If your container listens on the wrong port, the deploy "succeeds" but the health check never passes and the host keeps restarting it. Read the platform's port convention and bind to it (use `0.0.0.0`, note 06).

Endpoints and scaling basics

Once deployed, your **endpoint** is a public URL. Real usage brings capacity questions:

- **Vertical scaling** — a bigger machine (more CPU/RAM). Simple, but has a ceiling and a single point of failure.
- **Horizontal scaling** — more **replicas** (identical containers) behind a **load balancer** that spreads requests. This is the standard approach and why *stateless* services matter: any replica can serve any request because the model is loaded from the same artifact and nothing is kept in local memory between requests.
- **Autoscaling** — the platform adds/removes replicas based on load (CPU, request rate). Managed hosts do this for you; free tiers cap it low.

```
client --> load balancer
                        +--> [replica 1]
                        +--> [replica 2]  (each identical, stateless)
                        +--> [replica 3]
```

Why this matters: The design rule that makes scaling possible is *statelessness*. If your API stored per-user data in a Python dict in memory, replica 2 wouldn't know what replica 1 knew, and load balancing would break. Load the model, keep no session state in memory, and you can scale to N replicas trivially. This is why the FastAPI service in note 05 loads the model at startup and holds nothing else.

Cold starts

Free tiers **sleep** idle services to save resources. The first request after a sleep must wait for the container to wake, load the model, and start — the **cold start**. On free hosts this can be several seconds to tens of seconds; subsequent requests are fast (the service is "warm").

```
Idle service --- first request --> [spin up container + load model: 5-30s] --> response
(COLD)
Warm service --- next request --> [~50ms] --> response
(WARM)
```

Mitigations: - Keep the image small and the model load fast (note 06) so cold starts are shorter. - A scheduled ping (e.g., a cron/GitHub Action hitting `/health` every few minutes) keeps a free service warm — use sparingly and within the host's terms. - On paid tiers you'd set a minimum number of always-on instances; on free tiers, accept some cold starts and mention them as a known limitation.

Instructor insight: In the capstone demo, warm up your service by hitting `/health` a few times *before* you present — otherwise the first `/predict` in front of the class takes 20 seconds and looks broken. Cold starts are a real production consideration (they hurt the p99 latency from note 05), and knowing they exist is exactly the kind of practical detail interviewers probe.

Environment configuration and secrets

Your service needs config that differs per environment (dev vs. prod) and secrets that must never be public. The rule: **config comes in at runtime via environment variables; secrets are stored in the platform's secret manager — never in code, never in the image.**

```
# app.py -- read config from the environment, with safe defaults
import os
MODEL_STAGE = os.getenv("MODEL_STAGE", "Production")
LOG_LEVEL    = os.getenv("LOG_LEVEL", "info")
API_KEY      = os.getenv("API_KEY")      # secret, injected by the host, never hard-coded
```

- **In development:** a local `.env` file (added to `.gitignore` and `.dockerignore`).
- **In CI:** GitHub repository Secrets (note 07).
- **In production:** the host's environment/secret settings (Render "Environment", HF Spaces "Secrets").

Watch out — a leaked secret is public forever. If you commit an API key to a public repo, treat it as compromised the moment you push, even if you delete it later (it's in the Git history and may already be scraped). Rotate it immediately. Keep secrets out of code, out of the image, and out of client-visible responses/logs. This is both a security and a professionalism signal in interviews.

The deployment picture

```

GitHub repo (Dockerfile + app + model)
  | push to main (CD from note 07)
  v
Free host builds the image -> runs container -> HTTPS endpoint
  |                                     ^ reads MODEL_STAGE, API_KEY from host env/secrets
  |                                     | health check hits /health
  v
Public URL -> clients POST /predict -> predictions
           (cold start on first hit after idle; warm & fast after)

```

Mini-FAQ

Q: HF Spaces or Render for the capstone? A: Either is free and fine. Choose Render if you want push-to-deploy CD tied to GitHub and a plain web-service model. Choose HF Spaces if you like the ML-community context and a simple Docker Space. Both give an HTTPS URL; just mind the port convention.

Q: My deploy is stuck "waiting for health check." What's wrong? A: Almost always the port. The host is pinging a port your app isn't listening on, or the app bound to `127.0.0.1` instead of `0.0.0.0`. Match the host's expected port, bind `0.0.0.0`, and make sure `/health` returns 200 quickly without loading heavy resources.

Q: How do I update the deployed model without redeploying code? A: Load by MLflow registry *stage* (note 03): `models:/churn-model/Production`. Promote a new version to Production in the registry and your service picks it up (on restart or on a refresh), no image rebuild needed. That's the clean separation between shipping code and shipping models.

SESSION 9

09 — Monitoring, Drift & Governance

Learning objectives

By the end of this session you should be able to: - Distinguish **data drift** from **concept drift** with concrete examples. - List the **metrics** a deployed model needs monitored and how **alerting** works. - Explain drift detection conceptually with **PSI** and the **KS test**. - Design **retraining triggers**, and describe **model governance, lineage, and responsible ML**.

Key terms

- **Monitoring** — continuously observing a live system's health and quality.
- **Data drift** — the input distribution changes over time.
- **Concept drift** — the relationship between inputs and the target changes.
- **PSI (Population Stability Index)** — a number summarizing how much a distribution shifted.
- **KS test (Kolmogorov-Smirnov)** — a statistical test comparing two distributions.
- **Retraining trigger** — the condition that kicks off a new training cycle.
- **Lineage** — the traceable history of a model: data, code, and versions that produced it.
- **Governance** — the policies and records that keep ML accountable and compliant.

Why monitoring is the point of MLOps

A model is the only kind of software that can **degrade while the code is unchanged**. Nobody edits it, no bug is introduced, yet its accuracy quietly falls because the *world* changed. Recall note 01: models fail *after* training. Monitoring is how you notice — before your users (or your boss) do. Without monitoring, you're flying blind; the model could have been useless for three months and you'd only find out when someone complains.

Monitor two layers: 1. **Operational health** (same as any service): request rate, error rate, latency (p50/p95/p99 — note 05), CPU/memory, uptime. 2. **ML quality** (unique to models): input drift, prediction distribution, and — when labels eventually arrive — live accuracy.

Data drift vs. concept drift

Both hurt accuracy, but they're different problems:

- **Data drift** — the *inputs* change distribution, even if the underlying relationship holds. Example: your churn model was trained on customers aged 30-45; a new marketing campaign brings in mostly 18-24 year-olds. The model now sees inputs unlike anything it learned from. $P(X)$ changed.
- **Concept drift** — the *relationship* between inputs and target changes. Example: before a recession, high income strongly predicted low churn; during the recession, even high earners cut spending, so the old rule weakens. Same inputs, different meaning. $P(y|X)$ changed.

Data drift:	input distribution moves	-> $P(X)$ changes, $P(y X)$ same
Concept drift:	the rule itself moves	-> $P(y X)$ changes

Why this matters: The fix differs. Data drift often means "retrain on recent data that includes the new population." Concept drift means "the old patterns are wrong now — retrain, and maybe rethink features." Misdiagnosing wastes effort. And note the practical asymmetry: you can detect **data drift immediately**

(you have the inputs), but **concept drift needs labels**, which often arrive weeks later (did the customer actually churn?). So input-drift monitoring is your early-warning system.

Metrics and alerting

You track metrics over time and **alert** when one crosses a threshold or shifts sharply.

```
# Log per-prediction signals to a store/log for later analysis
import json, time, logging
logger = logging.getLogger("monitor")

def log_prediction(features: dict, prob: float, latency_ms: float):
    logger.info(json.dumps({
        "ts": time.time(),
        "features": features,      # to compute input drift later
        "prediction": prob,        # to watch the prediction distribution
        "latency_ms": latency_ms,
    })))
```

What to alert on: - **Operational**: error rate > 1%, p99 latency > target, service down. - **Prediction distribution shift**: e.g., the model suddenly predicts "will churn" for 60% of traffic when the historical rate is 15% — a strong sign something upstream broke (a feature pipeline change, note 01's data dependencies). - **Input drift**: a feature's distribution moved beyond a threshold (PSI/KS below). - **Accuracy drop**: once labels arrive, live F1/accuracy falls below the gate.

Watch out — alert fatigue. If you alert on every tiny wiggle, people mute the alerts, and a real fire goes unnoticed. Set thresholds that reflect *actionable* change (e.g., PSI > 0.2, error rate > 1% sustained for 5 minutes), and prefer a few meaningful alerts over dozens of noisy ones. An ignored alert channel is worse than none.

Drift detection, conceptually

You don't need heavy math to reason about drift. Two standard tools:

PSI — Population Stability Index

Compares a feature's distribution now (recent window) against a reference (training data), by bucketing values and measuring how the proportions moved. Rule-of-thumb reading: - **PSI < 0.1** — no significant shift. - **0.1-0.2** — moderate shift, investigate. - **> 0.2** — significant shift, likely retrain.

```
import numpy as np

def psi(reference, current, bins=10):
    # bucket by reference quantiles, compare proportions
    edges = np.quantile(reference, np.linspace(0, 1, bins + 1))
    edges[0], edges[-1] = -np.inf, np.inf
    ref_pct = np.histogram(reference, edges)[0] / len(reference)
    cur_pct = np.histogram(current, edges)[0] / len(current)
    ref_pct = np.clip(ref_pct, 1e-6, None)      # avoid divide-by-zero
    cur_pct = np.clip(cur_pct, 1e-6, None)
    return float(np.sum((cur_pct - ref_pct) * np.log(cur_pct / ref_pct)))

# psi(train_income, last_week_income) -> 0.27 => significant drift, investigate/retrain
```

KS test — Kolmogorov-Smirnov

A statistical test that compares two continuous distributions and returns a p-value. A small p-value (say < 0.05) says "these two samples likely come from different distributions" — i.e., drift.

```
from scipy.stats import ks_2samp
stat, p_value = ks_2samp(train_income, last_week_income)
if p_value < 0.05:
    print("Distribution shift detected for 'income'")
```

Why this matters: PSI gives an interpretable magnitude ("how much did it move?") with handy thresholds; the KS test gives a statistical yes/no. Together they turn a vague worry ("is the data different now?") into a number you can alert on and act on. Tools like Evidently AI (free, open-source) compute these for you and produce drift dashboards — worth mentioning, but understanding the concept is what matters.

Watch out — drift is not always bad. A detected input shift doesn't automatically mean the model is worse; sometimes the model generalizes fine. Drift is a *signal to investigate*, not an automatic verdict. Confirm with accuracy (when labels arrive) before assuming the model failed — and don't retrain reflexively on noise.

Retraining triggers

When should you retrain? Three common triggers: 1. **Scheduled** — retrain every week/month regardless. Simple, predictable; may retrain needlessly or too late. 2. **Performance-based** — retrain when live accuracy drops below a threshold. Ideal, but needs labels. 3. **Drift-based** — retrain when PSI/KS crosses a threshold. Acts early, before accuracy visibly falls, using only inputs.

Whatever the trigger, the retrained model **must pass the same CI gates** (note 07) — accuracy threshold and champion/challenger comparison against current Production — before promotion in the registry (note 03). Automatic retraining without gates is dangerous: you could auto-deploy a worse model.

```
Drift alert (PSI>0.2) OR accuracy<0.75 OR monthly schedule
-> retrain on latest versioned data
-> run data/behavioral tests + accuracy gate + champion/challenger
-> if better: register new version, promote to Production
-> if not:    keep current model, raise a human review ticket
```

Instructor insight: The loop only closes when you can get *labels*. For churn you learn the truth weeks later; for fraud, sometimes never (or via chargebacks). A huge, under-taught part of real MLOps is **label collection and delay** — designing how ground truth flows back so you can actually measure live accuracy. In interviews, mentioning "how do labels arrive, and how delayed are they?" instantly marks you as someone who's thought past the demo.

Governance, lineage, and responsible ML

As models drive real decisions (loans, hiring, healthcare — increasingly regulated in India and worldwide), you must be able to *account* for them:

- **Lineage** — for any prediction, trace back: which model version, which training run, which data version, which code commit. The MLflow registry + data manifests (notes 02, 03) give you this. When a decision is challenged, you can reconstruct exactly how it was made.

- **Governance** — the policies and records: who approved promoting a model to Production, what tests it passed, when, and why. An audit trail, not vibes.
- **Responsible ML** — fairness (does the model perform equally across groups like gender/region?), transparency (can you explain a decision?), privacy (are you handling personal data lawfully?), and human oversight for high-stakes decisions.

```
# Lineage metadata attached to every deployed model version
lineage = {
    "model_version": "churn-model:v4",
    "training_run": "mlflow://run/a1b2c3",
    "data_sha256": "9f2c...",
    "git_commit": "e4f5a6",
    "promoted_by": "venkat",
    "promoted_at": "2026-07-17",
    "gate_metrics": {"f1": 0.81, "beats_prod": True},
}
```

Why this matters: Governance is not bureaucracy for its own sake — it's what lets an organization *trust* an automated decision-maker. If a customer disputes a rejected loan, "the model said so" is not an answer; "version v4, trained on this audited data, tested for fairness across regions, approved on this date" is. Building lineage in from the start (which our whole pipeline does) makes this almost free.

Mini-FAQ

Q: If I can't get labels quickly, is monitoring pointless? A: No. Input drift (PSI/KS) and prediction-distribution monitoring need *no labels* and catch most problems early — broken pipelines, shifting populations. Labels give you the final accuracy truth, but proxy monitoring is your fast early-warning system.

Q: PSI says drift but accuracy (once labels came) is fine. Do I retrain? A: Not automatically. Drift is a signal, not a sentence. If accuracy holds, note it and keep watching. Retrain when quality actually suffers or drift is severe and sustained. Don't chase noise.

Q: Isn't governance overkill for a student project? A: The heavy compliance is. But the *lineage habit* — recording data version, run, commit, and metrics for each model — costs almost nothing (you already produce all of it) and is exactly what makes a capstone look professional and an interview answer credible.

SESSION 10

10 — Capstone & Interview Prep

Learning objectives

By the end of this session you should be able to: - Assemble everything from notes 01-09 into one **end-to-end MLOps project**. - Explain how each component connects and hands off to the next. - Present the project as a coherent story (the way you would in an interview or demo). - Answer common **MLOps interview questions** with strong, specific answers.

Key terms

- **End-to-end pipeline** — the full path from data to a monitored, deployed model.
- **Capstone** — your integrating project demonstrating all course skills.
- **Mock technical interview** — the assessment format; you explain and defend your choices.

How the whole thing fits together

Everything in this course is one pipeline. Each note was a stage; here is the assembled system, all on free tools:

```
[ Data ] --version + hash (note 02)--> data_manifest.json, dated snapshots
|
[ Feature pipeline ] --shared features.py (note 02)--> used by BOTH train & serve (no skew)
|
[ Train + experiment tracking ] --MLflow (note 03)--> params, metrics, artifacts per run
|
[ Model selection + registry ] --MLflow Registry (note 03)--> churn-model v4, stage=Production
|
[ Serialize + package ] --joblib/bundle + pinned deps (note 04)--> reproducible artifact
|
[ Serve ] --FastAPI + Pydantic (note 05)--> /health, /predict, validated I/O
|
[ Containerize ] --Dockerfile (note 06)--> churn-api:sha, identical everywhere
|
[ CI/CD ] --GitHub Actions (note 07)--> lint + unit/data/behavioral tests + F1 gate + build + deploy
|
[ Deploy ] --Render / HF Spaces (note 08)--> live HTTPS endpoint, config via env/secrets
|
[ Monitor ] --drift (PSI/KS) + latency + accuracy (note 09)--> alerts
|
[ Retrain trigger ] ----- loops back to Data -----^
```

The capstone deliverable is a **GitHub repo** that contains all of this and ends in a **live URL** you can demo, plus an MLflow UI showing your runs and registry.

Instructor insight: The single most impressive thing you can show is not a high accuracy number — it's the *loop closing*. "Here's my model live at this URL; here's the MLflow run and data hash that produced it; here's the CI gate that would block a bad retrain; here's how a drift alert would trigger a new cycle." That

narrative — reproducible, tested, deployed, monitored — is what separates "I trained a model" from "I built an ML system." Aim your capstone at telling that story cleanly, even if each piece is small.

A realistic capstone scope (15 hours)

Keep it small and complete rather than large and half-built. A strong minimal capstone:

- A simple tabular dataset (e.g., churn, or a UCI classification set) — versioned with a hash/manifest.
- One `features.py` imported by both trainer and API.
- 3-5 tracked MLflow runs; best model registered and promoted to Production.
- A FastAPI app with `/health` and `/predict`, Pydantic-validated.
- A `Dockerfile` that builds and runs locally.
- A GitHub Actions workflow: lint + `pytest` (a data test + a behavioral test + an F1 gate) + docker build.
- Deployed to Render or HF Spaces with a working public URL.
- A short README explaining the pipeline and one drift check (PSI on one feature) demonstrated in a notebook.

Why this matters: Graders and interviewers reward *completeness of the loop* over raw model performance. A 78%-accuracy model that is versioned, tested, deployed, and monitored beats a 95% model living in a notebook. Depth of the pipeline is the assessed skill in this course.

Watch out — leaving the demo cold. Free hosts sleep (note 08). Before your assessment, hit `/health` a few times to warm the service, and have a backup screen recording in case the network fails. A 25-second cold start in front of the panel reads as "broken." Rehearse the demo end-to-end at least once.

MLOps interview-prep primer

The course is assessed partly by a mock technical interview. Below are common questions with the shape of a strong answer. Practice saying these out loud — specific and concrete beats vague and buzzword-y.

Q1. What is MLOps, and how is it different from DevOps?

MLOps is DevOps for ML plus data. DevOps versions and ships code; MLOps must version code, data, and models together, and it adds two problems DevOps never had: models silently degrade as the world changes (drift), and training-serving skew. So beyond CI/CD it needs experiment tracking, a model registry, and drift monitoring.

Q2. Why do models fail in production even when training accuracy was high?

Four common reasons: data/concept drift (the world changed), train/serve skew (features computed differently in serving), upstream data pipeline breakage, and no reproducibility so you can't debug. The dangerous ones are silent — the API returns 200 OK while predictions are wrong.

Q3. What is training-serving skew and how do you prevent it?

When features at serving time differ from training time — different code, time windows, or null handling. Prevent it by putting feature logic in one shared module used by both trainer and API, validating the input schema (names, order, dtypes) at the serving boundary, and ideally using a feature store so one definition feeds both.

Q4. How do you make an ML experiment reproducible?

Four pillars: same data (version it with a hash/DVC), same code (git commit), same environment (pinned `requirements.txt`, ideally a Docker image), and same seeds (fix numpy/random/framework seeds). If any one is missing, results can differ.

Q5. Walk me through deploying a model to production.

Serialize and package it (joblib bundle or MLflow model with pinned deps), wrap it in a FastAPI service with health and inference endpoints and Pydantic validation, containerize with Docker so the environment is reproducible, gate it through CI/CD (lint, tests, data tests, an accuracy threshold), deploy the image to a host, then monitor latency, errors, and drift, with a retraining trigger closing the loop.

Q6. Data drift vs. concept drift?

Data drift is a change in the input distribution $P(X)$; concept drift is a change in the input-target relationship $P(y|X)$. You can detect data drift immediately from inputs (PSI, KS test); concept drift needs labels, which often arrive late. Data drift often means retrain on recent data; concept drift may need rethinking features.

Q7. How do you test an ML model in CI?

Beyond normal unit tests: data tests (schema, ranges, null rate), behavioral tests (directional/invariance/min-functionality checks on crafted inputs), and an accuracy gate — the model must beat a threshold and ideally the current Production model (champion/challenger) or the pipeline refuses to deploy. Fix seeds so gates aren't flaky.

Q8. What is a model registry and why use stages?

A versioned catalog of models with lifecycle stages (None/Staging/Production/Archived). Serving loads by stage, not filename, so promoting a new model or rolling back is a stage transition — no code change. It also gives an audit trail of who promoted what and when.

Q9. Containers vs. VMs — why Docker for ML?

VMs virtualize hardware and run a full guest OS (heavy); containers virtualize the OS and share the host kernel (light, seconds to start). Docker packages the exact environment — OS libs, pinned Python deps, model — so it runs identically everywhere, solving "works on my machine." That's why hosts run containers.

Q10. How do you monitor a deployed model, and when do you retrain?

Monitor operational metrics (error rate, p95/p99 latency, uptime) and ML metrics (input drift via PSI/KS, prediction distribution, and live accuracy once labels arrive). Retrain on a trigger: schedule, accuracy drop, or drift threshold — and always re-run the CI gates before promoting the new model.

Q11. A model's p99 latency spiked. How do you debug?

Separate model time from everything around it. Check: is the model being reloaded per request instead of at startup? Did feature code start hitting a slow dependency? Did input size grow? Is the host cold-starting? Measure percentiles and add timing to isolate the stage.

Q12. Is `pickle` safe? What are the alternatives?

No — unpickling executes code, so never load untrusted pickles. Use joblib for sklearn (same caveats, better for arrays), and ONNX when you need cross-language serving or speed. Always load with the same library versions you saved with.

Instructor insight: In the mock interview I care less about textbook definitions and more about *trade-off reasoning*. "Why FastAPI over Flask?" "Why joblib over ONNX here?" "Why managed over self-managed?" There's rarely one right answer — the signal is whether you can name the trade-off and justify a choice for *this* context. Always tie it back to your capstone: "In my project I chose X because Y." Concrete beats abstract every time.

Mini-FAQ

Q: I'm nervous about the mock interview. What should I over-prepare? A: The end-to-end walkthrough (Q5) and your own capstone. If you can narrate your repo — data versioning, MLflow runs, the FastAPI service, the Dockerfile, the CI gate, the live URL, the drift check — you can answer 80% of questions by pointing at real work you did.

Q: What if I'm asked something we didn't cover? A: Reason out loud from principles. Say what you'd measure, what the trade-off is, and how you'd validate. Demonstrated thinking beats a memorized answer to the wrong question. "I haven't used that, but here's how I'd approach it" is a strong answer.

Q: How do I make my capstone stand out? A: Close the loop and show lineage. A live URL plus an MLflow registry plus a CI accuracy gate plus one demonstrated drift metric tells a complete, professional story. Polish the README and rehearse a 3-minute demo. Completeness and clarity win.

REFERENCE

MLOps Cheat Sheet — One-Page Quick Reference

MLOps and Model Deployment · Vel Tech · Dr. Venkateshwaran (Venkat) Loganathan

All tools below are free and run locally or on free hosts.

The end-to-end pipeline (text diagram)

```

[DATA] --version(hash/DVC)--> [FEATURES features.py] --shared by both--> [TRAIN + MLflow
tracking]
                                                                    |
                                                                    best run -> [MLflow REGISTRY: v4 =
Production]
                                                                    |
[SERIALIZE joblib/ONNX + pinned deps] <-----
|
[SERVE FastAPI: /health /predict + Pydantic]
|
[CONTAINER Dockerfile -> image:sha]
|
[CI/CD GitHub Actions: lint + tests + data tests + F1 gate + build + deploy]
|
[DEPLOY Render / HF Spaces -> HTTPS URL]
|
[MONITOR latency + errors + drift(PSI/KS) + accuracy] --alert--> [RETRAIN trigger]
|
+----- loops back to DATA -----+

```

Git

```

git init
git add . && git commit -m "message"
git push origin main
git checkout -b feature-x          # new branch
git log --oneline                  # find a commit for lineage

```

MLflow

```

pip install mlflow
mlflow ui                          # UI at http://127.0.0.1:5000 (local ./mlruns)

```

```
import mlflow, mlflow.sklearn
mlflow.set_experiment("churn-model")
with mlflow.start_run(run_name="rf"):
    mlflow.log_params({"max_depth": 8})
    mlflow.log_metric("f1", 0.81)
    mlflow.log_artifact("data_manifest.json")
    mlflow.sklearn.log_model(model, "model")

# Registry
mlflow.register_model("runs:/<RUN_ID>/model", "churn-model")    # -> version N
# transition: client.transition_model_version_stage(name, version, "Production")
model = mlflow.pyfunc.load_model("models:/churn-model/Production") # load by STAGE
```

Reproducibility

```
python --version          # record it
pip freeze > requirements.txt # pin exact versions
pip install -r requirements.txt # reproduce anywhere
```

```
import numpy as np, random
random.seed(42); np.random.seed(42)    # + torch/tf seed for DL
```

Serialize

```
import joblib
joblib.dump(model, "model.joblib")      # preferred for sklearn
model = joblib.load("model.joblib")
# pickle: built-in but INSECURE (never load untrusted); ONNX: cross-language/speed
```

FastAPI + uvicorn

```
pip install fastapi uvicorn pydantic
uvicorn app:app --host 0.0.0.0 --port 8000 --reload    # docs at /docs
```

```
from fastapi import FastAPI
from pydantic import BaseModel, Field
app = FastAPI()
model = joblib.load("model.joblib")          # load ONCE at startup
@app.get("/health")
def health(): return {"status": "ok"}
class Req(BaseModel):
    income: float = Field(gt=0)
@app.post("/predict")
def predict(r: Req): ...
```

Docker

```
docker build -t churn-api:1.0 .
docker run -d -p 8000:8000 -e MODEL_STAGE=Production --name churn churn-api:1.0
docker ps          # list running
docker logs churn  # view logs
docker stop churn && docker rm churn
```

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .           # copy deps FIRST (layer cache)
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"] # 0.0.0.0!
```

GitHub Actions (free for public repos)

```
# .github/workflows/ci.yml
on: { push: { branches: [main] }, pull_request: {} }
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.11" }
      - run: pip install -r requirements.txt
      - run: ruff check .
      - run: pytest -q           # includes data tests + F1 gate
# secrets: ${ secrets.NAME } -- NEVER hard-code keys
```

Drift (concepts)

```
# PSI: <0.1 stable | 0.1-0.2 investigate | >0.2 retrain
# KS test:
from scipy.stats import ks_2samp
stat, p = ks_2samp(train_feature, recent_feature) # p<0.05 => drift
```

Vocabulary

TERM	ONE-LINE MEANING
MLOps	DevOps for ML + data (build, deploy, monitor, maintain models)
ML lifecycle	Data -> features -> train -> deploy -> monitor -> loop back
Technical debt (ML)	Hidden future cost: feedback loops, glue code, data dependencies
Maturity 0/1/2	Manual / automated training pipeline / full CI/CD+CT
Reproducibility	Same data + code + env + seed = same result
Data versioning	Snapshot/hash the exact dataset used
Feature pipeline	Code turning raw data into model inputs
Train/serve skew	Features differ between training and serving (silent bug)
Feature store	Central system serving consistent features to train & serve
Experiment tracking	Recording params/metrics/artifacts per run (MLflow)
Run / Experiment	One training execution / a named group of runs

TERM	ONE-LINE MEANING
Model Registry	Versioned model catalog with stages
Stage	None / Staging / Production / Archived
Serialization	Saving a model to disk (pickle/joblib/ONNX)
joblib	Preferred sklearn serializer (arrays)
ONNX	Cross-language, fast model format
"Works on my machine"	Env mismatch; fixed by pinning + Docker
Batch vs online	Bulk offline scoring vs low-latency live scoring
FastAPI / Pydantic	Serving framework / input validation
Endpoint	A URL + method the API answers (/health, /predict)
Latency (p50/p95/p99)	Response time percentiles
Container vs VM	Shares host kernel (light) vs full guest OS (heavy)
Image / layer / tag	Blueprint / cached build step / version label
Port mapping	-p host:container
CI / CD	Auto build+test / auto release
Data test	Schema/range/null checks on data
Behavioral test	Directional/invariance/min-functionality checks on model
Accuracy gate	Min metric to allow deploy (+ champion/challenger)
Continuous Training (CT)	Automated retraining on new data
Self-managed vs managed	You run the server vs platform runs it
Cold start	Delay to wake an idle service
Env var / secret	Runtime config / sensitive value (never committed)
Data drift	$P(X)$ changes (input distribution)
Concept drift	$P(y X)$ changes (input-target relationship)
PSI / KS test	Drift magnitude / statistical drift test
Retraining trigger	Schedule / accuracy drop / drift threshold
Lineage	Traceable data+code+version behind a model
Governance	Policies + audit trail for accountable ML
...	